Assignment-6.4
Name:A.Vignesh
Hall ticket no:2303A51640
Batch:09
Lab:6

Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals
Lab Objectives:
Week1 -
Wednesday
The objectives of this lab are to:
• To explore AI-powered auto-completion features for core Python constructs.
• To analyze how AI suggests logic for class definitions, loops, and conditionals.
• To evaluate the completeness and correctness of code generated by AI assistants.
Lab Outcomes (LOs):
• After completing this lab, students will be able to:
• Use AI tools to generate and complete class definitions and methods.
• Understand and assess AI-suggested loops for iterative tasks.
• Generate conditional statements through prompt-driven suggestions.
• Critically evaluate AI-assisted code for correctness and clarity.

task 1: Student Performance Evaluation System

Scenario
Build a simple academic management module for a university system where student performance needs to be evaluated automatically.
Task Description
You are Create the skeleton of a Python class named Student with the attributes:
• name
• roll_number
• marks
Write only the class definition and attribute initialization.
Then, using GitHub Copilot, prompt the tool to complete:
• A method to display student details
• A method that checks whether the student's marks are above the class average and returns an appropriate message

Use comments or partial method names to guide Copilot for code completion.

Expected Outcome

• A completed Student class with Copilot-generated methods

• Proper use of:

o self attributes

o Conditional statements (if-else)

• Sample output showing student details and performance status

prompt:Build a simple academic management module for a university system where student performance needs to be evaluated automatically.

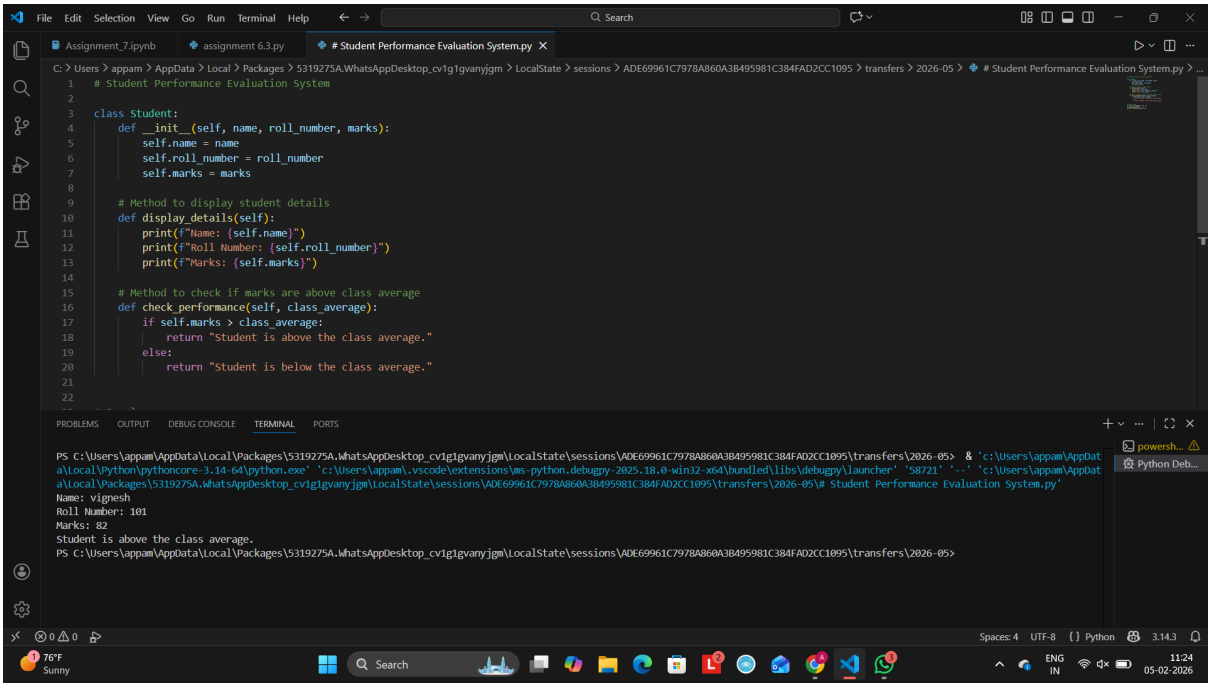
Create the skeleton of a Python class named Student with the attributes:

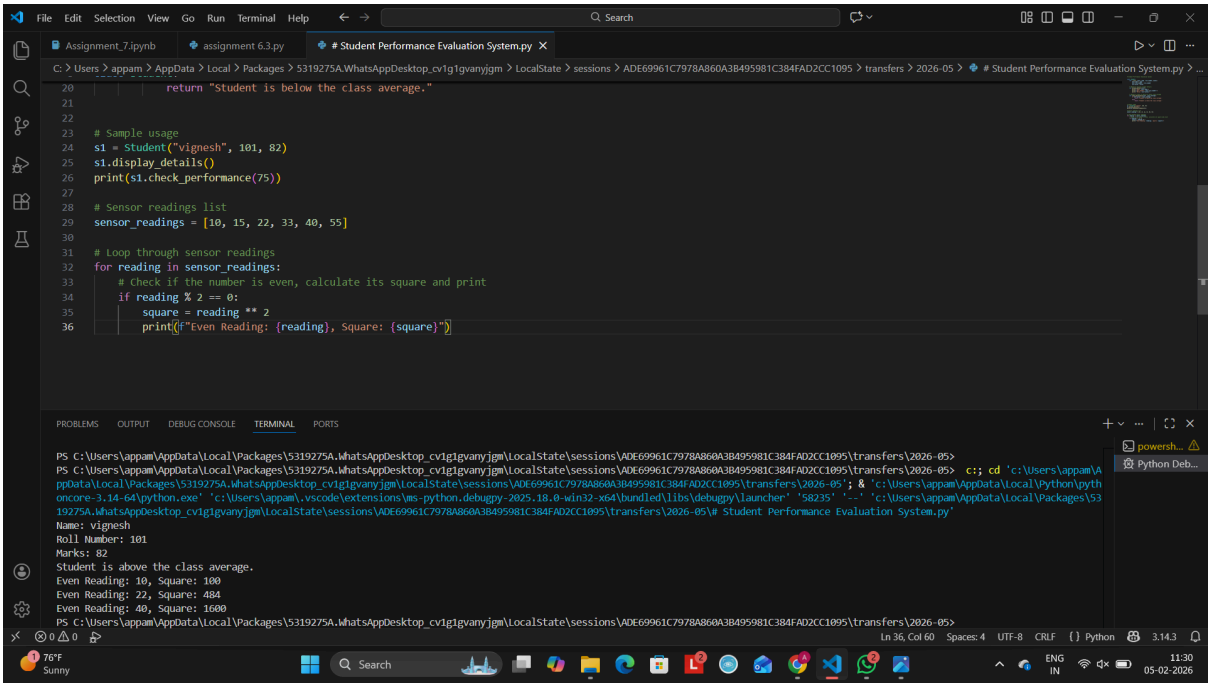
• name

• roll_number

• marks

Code & output:





Analysis:

Student Performance Evaluation System

This task focuses on basic **object-oriented programming** in Python. It evaluates understanding of class structure, attribute initialization, and method usage. Conditional logic is used to compare a student's marks with the class average, reinforcing the use of `if-else` statements and `self` attributes. GitHub Copilot assists in completing methods based on comments.

Task 2: Data Processing in a Monitoring System

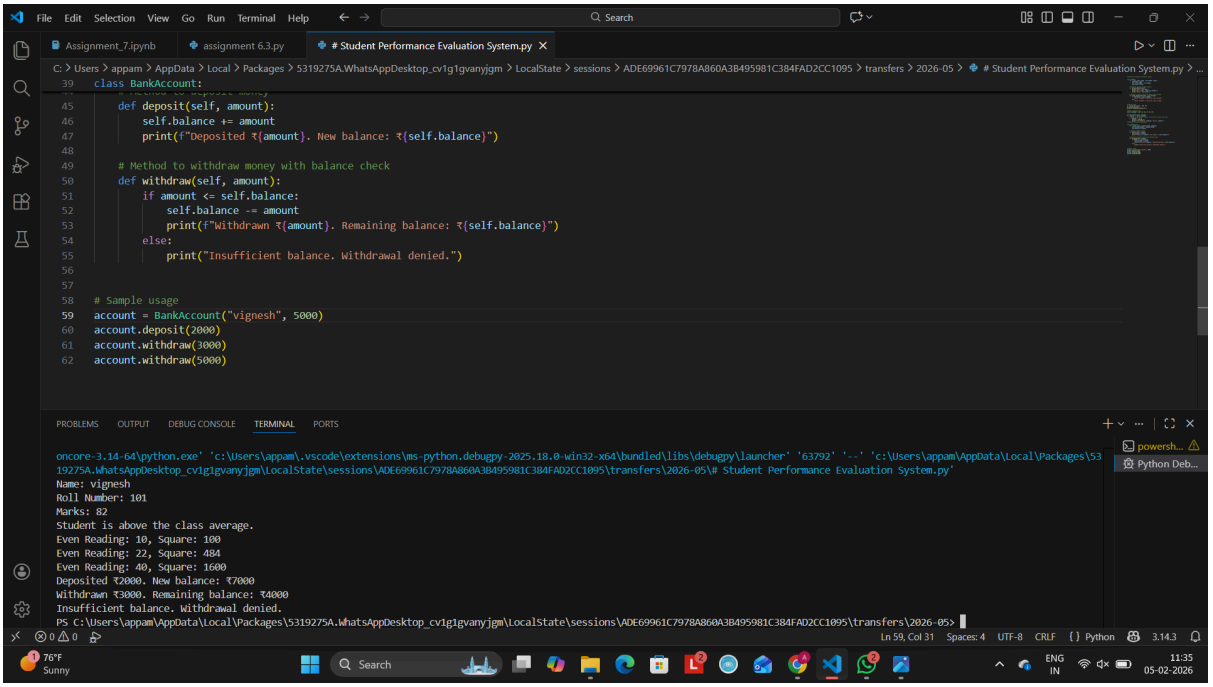
Scenario

You are working on a basic data monitoring script where sensor readings are collected as numbers. Only even readings need further processing.
Task Description
Write the initial part of a for loop to iterate over a list of integers representing sensor readings.
Add a comment prompt instructing GitHub Copilot to:
• Identify even numbers
• Calculate their square
• Print the result in a readable format
Allow Copilot to complete the remaining loop logic.
Expected Outcome
• A complete for loop generated by Copilot
• Use of:
o Modulus operator to identify even numbers
o Conditional statements
• Correct and formatted output for valid inputs

prompt:Data Processing in a Monitoring System

Identify even numbers
• Calculate their square
• Print the result in a readable format

code&output



Analysis:

Data Processing in a Monitoring System

This task emphasizes **looping and conditional checks**. It demonstrates how to iterate through a list of sensor readings, identify even numbers using the modulus operator, and perform calculations on valid data. Copilot is guided to complete logic that squares even numbers and formats output clearly.

Task 3: Banking Transaction Simulation

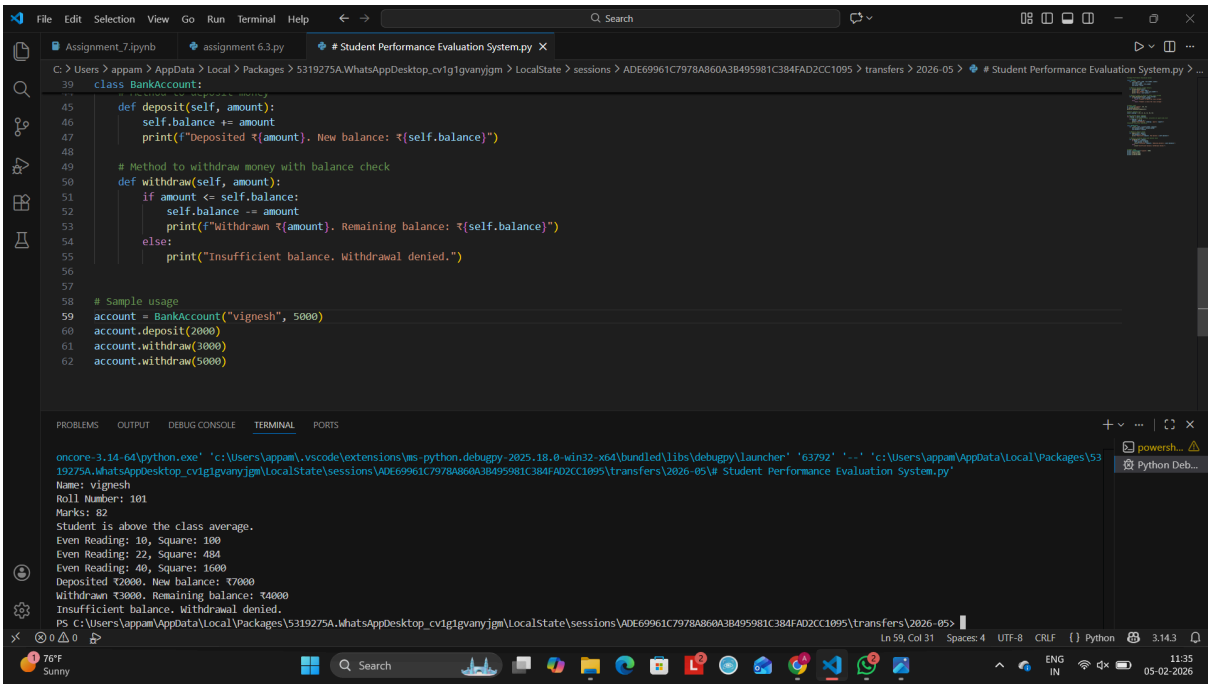
Scenario

You are developing a basic banking module that handles deposits and

withdrawals for customers.
Task Description
Create the structure of a Python class named BankAccount with attributes:
• account_holder
• balance
Use GitHub Copilot to complete methods for:
• Depositing money
• Withdrawing money
• Preventing withdrawals when the balance is insufficient
Guide Copilot using method names and short comments.
Expected Outcome
• A fully functional BankAccount class
• Copilot-generated methods using:
o if-else conditions
o Class attributes via self
• Proper handling of invalid withdrawal attempts with user-friendly Messages

Prompt:develop a basic banking module that handles deposits andwithdrawals for customers

code&output:



Analysis:

Banking Transaction Simulation

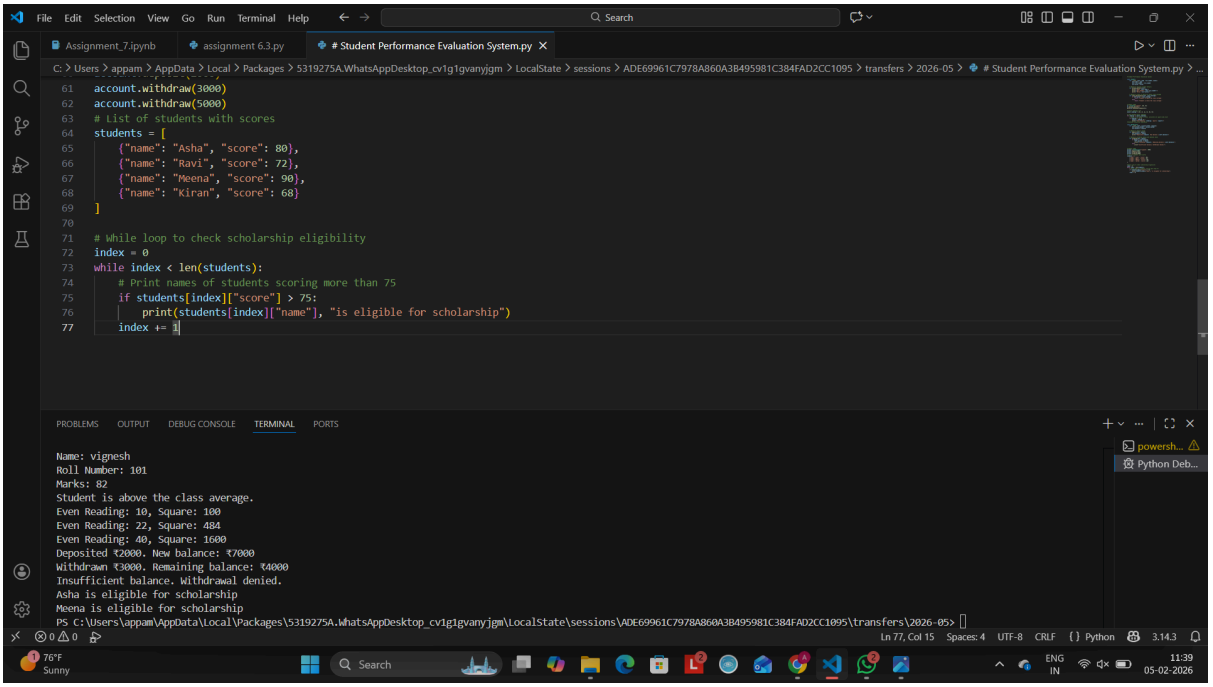
This task applies OOP concepts to a **real-world banking scenario**. It tests the ability to manage state using class attributes and handle deposits and withdrawals safely. Conditional statements ensure invalid withdrawals are prevented, highlighting error handling and user-friendly messaging.

Task 4: Student Scholarship Eligibility Check

Prompt:check whether the student is eligible or not for scholarship

Scenario
A university wants to identify students eligible for a merit-based scholarship based on their scores.
Task Description
Define a list of dictionaries where each dictionary represents a student with:
• name
• score
Write the initialization and list structure yourself.
Then, prompt GitHub Copilot to generate a while loop that:
• Iterates through the list
• Prints the names of students who scored more than 75
Use comments to guide Copilot's code completion.
Expected Outcome
• A complete while loop generated by Copilot
• Correct index handling and condition checks
• Cleanly formatted output listing eligible students

code&output:



Analysis:

Student Scholarship Eligibility Check

This task focuses on **data structures and loop control**. A list of dictionaries is used to store student data, and a `while` loop iterates through it to filter eligible students. It reinforces index management, condition checking, and clean output formatting with Copilot-generated logic.

Task 5: Online Shopping Cart Module

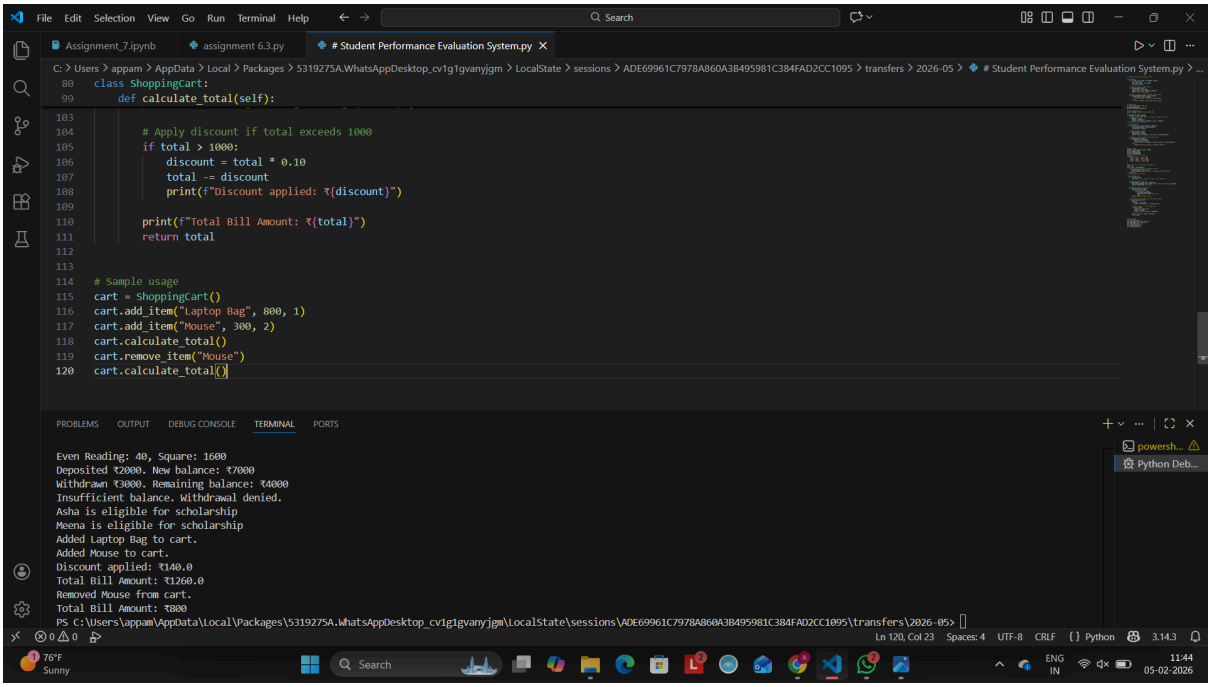
Scenario
You are designing a simplified shopping cart system for an e-commerce website that supports item management and discount calculation.
Task Description
Begin writing a Python class named ShoppingCart with:
• An empty list to store items (each item may include name, price, quantity)

Use GitHub Copilot to generate methods that:
• Add items to the cart
• Remove items from the cart
• Calculate the total bill using a loop
• Apply conditional discounts (e.g., discount if total exceeds a certain amount)
Use meaningful comments and method names to guide Copilot.
Expected Outcome
• A fully implemented ShoppingCart class
• Copilot-generated loops and conditional logic
• Correct handling of item addition, removal, and discount calculation
• Sample input/output demonstrating cart functionality

prompt: design a simplified shopping cart system for an e-commerce website that supports item management and discount calculation.

Write a Python class named ShoppingCart with:
• An empty list to store items (each item may include name, price,
quantity)

code&output:



Analysis:

Online Shopping Cart Module

This task integrates **lists, loops, conditionals, and OOP** to simulate an e-commerce cart system. It covers item management, total calculation, and discount application based on conditions. Copilot helps generate methods that demonstrate practical use of loops and decision-making logic.